DIATraceExtractor V0.2: release report (2026-10-12)

What it is. `DIATraceExtractor` is an OpenMS TOPP tool (C++23, OpenMS develop) that extracts DIA fragment and precursor-isotope traces (XICs) from a predicted spectral library and writes them as Parquet/TSV. It calibrates retention time and mass against the run itself, needs no identifications, and does no scoring. This report gathers what has been measured about the quality of its traces and its speed against DIA-NN 1.7.12 and OpenSWATH, the design decisions those measurements led to, and what is still open. Details live in the dossier (`00-dossier.md`), the extraction description (`EXTRACTION.md`) and the backlog (`BACKLOG.md`).

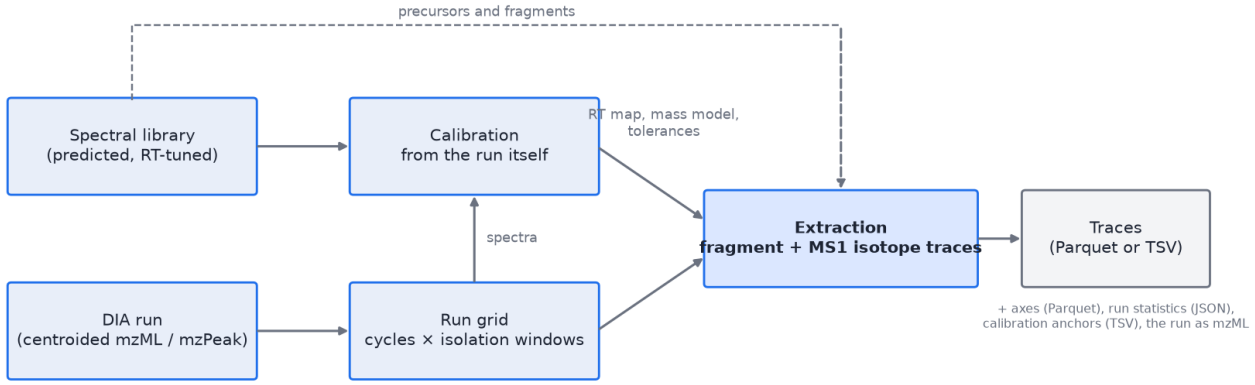
Verdict in one paragraph. At equal extraction rules the three tools produce the same traces; the differences between them are rule choices, and ours are the best-supported ones: a mass offset measured from the run (both instruments need it), a tolerance expressed in multiples of the measured error width, and a narrow, mass-corrected MS1 window that captures 98% of the signal at 1% interference where DIA-NN's export captures 92% at 5% and OpenSWATH 100% at 22%. The extraction runs 14–21× faster than DIA-NN and 40–100× faster than OpenSWATH on the same inputs, in 2.7 GiB, with byte-identical output at any thread count and from any of the three readers (fast mzML, OpenMS mzML, mzPeak). It runs end to end on a second instrument class (timsTOF diaPASEF) with the same agreement to DIA-NN as on the Orbitrap Astral. What remains open is one modelling question: the tolerance multiple is set per instrument class from the identification screen, not measured from the run, because the calibration anchors cannot see the interference that sets the optimum.

1. Data, libraries, references

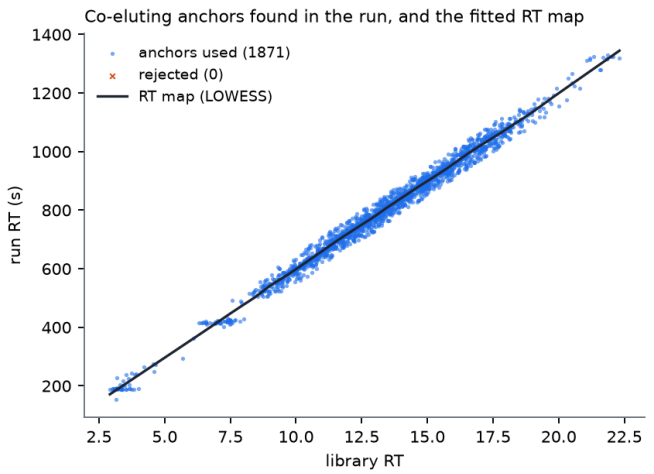
	Orbitrap Astral	**timsTOF SCP (diaPASEF)**
data set	ProteoBench DIA entrapment, 15 min, 50 ng, runs 01–03 (5.8 GB mzML each)	ProteoBench diaPASEF, conditions A and B, sample Alpha 01 (`.d`, converted to mzPeak and, by this tool, to mzML)
library	DIALibGen 0.11.1, human (3.5 M precursors) and entrapment (7.0 M); RT model tuned on DIA-NN IDs of run 01	DIALibGen, HYE mixed species, 4.15 M precursors (predicted RT, `-rt_mode auto`)
reference traces	DIA-NN 1.7.12 (last open-source version, `--xic` export patched), OpenSwathWorkflow (OpenMS develop)	DIA-NN 1.7.12 on our mzML export (1.7.12 reads no `.d`)
restriction for comparisons	DIA-NN's 1% FDR IDs (78–93 k precursors)	DIA-NN's IDs (54,557 A01; 55,665 B01)

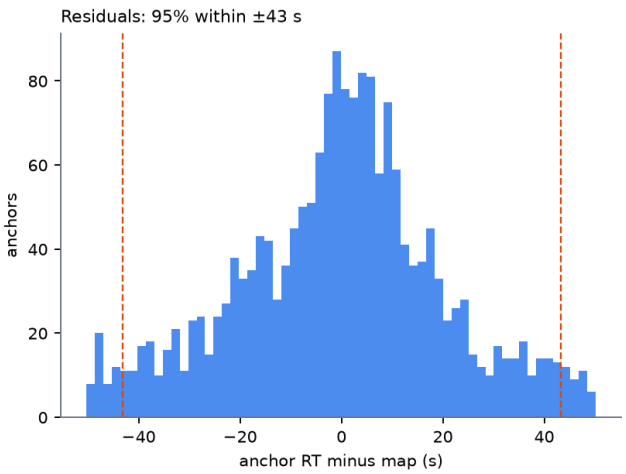
DIA-NN exports traces only while searching, so its times include the search; OpenSWATH's include its scoring.

2. How the extraction works



- **Run grid.** Every centroid of the run goes into one memory-mapped pool (m/z as uint32 at 1e-5 Th, intensity as float32: 8 bytes per peak), indexed by isolation window and DIA cycle, with MS1 on its own scan axis. Three readers feed it: a fast mzML parser, the OpenMS mzML reader, and the mzpeak-openms library for mzPeak files (diaPASEF frames are split per isolation window and collapsed across TIMS scans within 15 ppm).
- **Calibration** (`EXTRACTION.md` §3). A panel of 5% of the library (every k-th row, never the `-ids` subset) is searched for co-eluting fragment apexes: a strict first pass over the whole run yields anchors for a robust LOWESS RT map; a relaxed second pass within ±max(4σ, 30 s) of the map refines it. The apex centroids of the strict anchors give the mass model: a median offset, or a line offset + slope × m/z when the m/z dependence is material (new in V0.2), and the error width as 1.4826 × MAD of the residuals. Tolerances are multiples of that width.
- **Extraction.** Each precursor's fragments and MS1 isotopes are extracted within ±60 s of the mapped RT (or a measured window, `-rt_window 0`), by a merge join of the sorted target windows against each spectrum's centroids, taking the most intense centroid per window (DIA-NN's rule, `-aggregation max`) or their sum (OpenSWATH's, `sum`).
- **Output.** Traces as Parquet (`-out`, one row per trace, sharded) or as a long TSV (`-out_tsv`), with the axes file (`-out_axes`: global cycle axis, exact acquisition times per window, MS1 scan times), run statistics as JSON (`-out_stats`), the calibration anchors as TSV (`-out_calibration`, reusable as `-rt_anchors`), and the loaded run as a centroided mzML (`-out_mzml`, one spectrum per isolation window; how the diaPASEF runs reached DIA-NN).
- **Scheduling.** One worker pool runs library reading, extraction, encoding and writing as task classes rebalanced by pipeline pressure, with in-order issue and an ordered commit, so the output does not depend on the thread count.



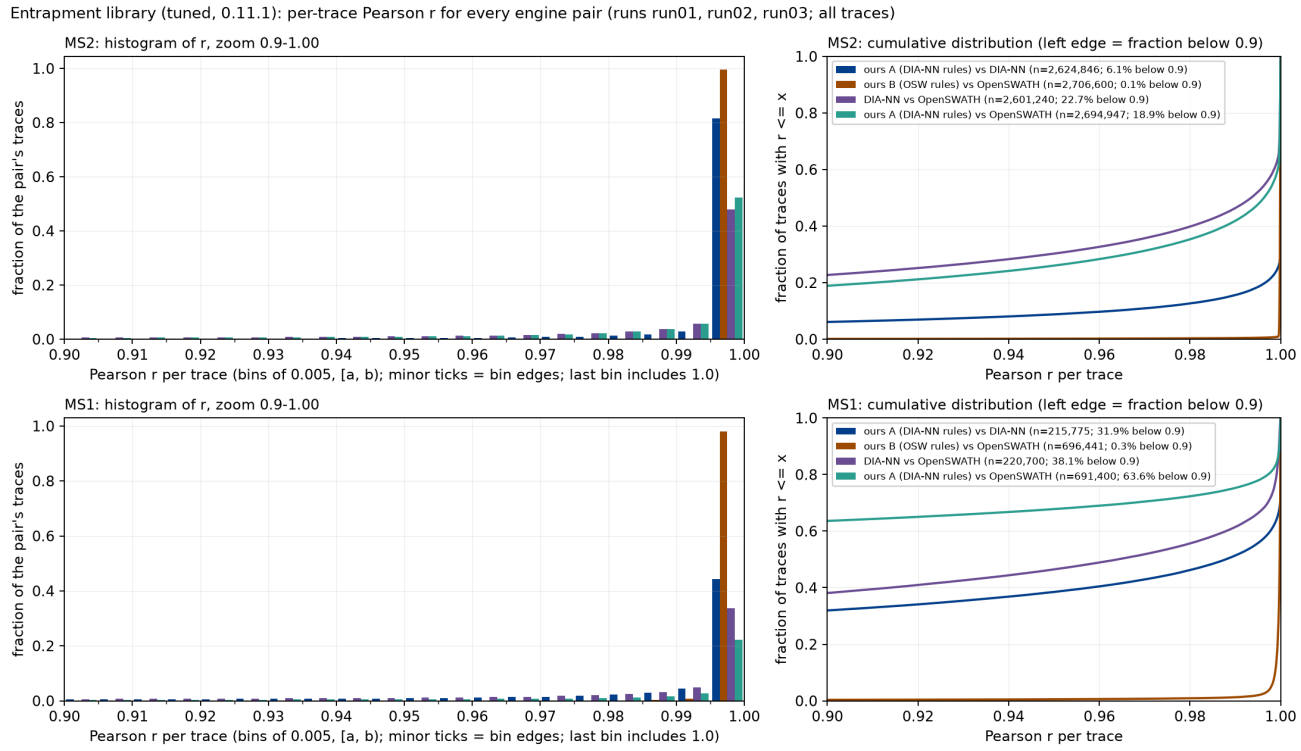


The two extraction schemes, A and B. The engines differ in a handful of rules, and the tool can follow either set, so that each comparison measures the rules and not the implementation:

rule	A (DIA-NN's rules; the default)	B (OpenSWATH's rules)
value per spectrum	the most intense centroid in the m/z window (`-aggregation max`)	the sum of all centroids in the window (`sum`)
mass correction	offsets (and the m/z-dependent term) measured from the run (`-mass_calibration auto`)	none: the window sits at the library m/z
tolerance	multiples of the measured widths (`-mz_sigma_ms2` , `-mz_sigma_ms1`), or fixed ppm	DIA-NN's optimised MS2 accuracy for both levels, as the benchmark gave OpenSWATH (`-mz_tol` , `-mz_tol_ms1`)
MS1 isotopes	M+k at m/z + k · 1.00336 / z	M+k at m/z + k · 1.00336 regardless of charge (`-ms1_isotope_spacing unit`)
RT window, MS1 axis	±60 s around the mapped library RT; MS1 at its own scans	the same

Scheme A with both tolerances set to DIA-NN's accuracy is called P (parity) in the dossier. Scheme B exists to show that the tool reproduces OpenSWATH's traces; its rules are not recommended (§4). Everything else is shared: the run grid, the RT map, the merge join, the output.

3. Are the traces the same? (Astral, shared tuned library, all DIA-NN IDs)



pair	MS2 median r	MS2 r ≥ 0.995	MS1 median r
ours B (OpenSWATH rules) vs OpenSWATH	0.9999	≈ 99%	0.9998
ours A (DIA-NN rules) vs DIA-NN	1.0000	81%	0.988
DIA-NN vs OpenSWATH	0.994	48%	0.964

Ours B reproduces OpenSWATH nearly exactly; ours A reproduces DIA-NN at MS2. The two reference engines disagree with each other more than either disagrees with its matching configuration of ours. The remaining MS2 tail against DIA-NN (3.5% of traces with r < 0.9 on run 01) is entirely its float32

quadratic window centre, shifted by about 0.5 ppm from ours: emulating that arithmetic reproduces every one of 35,000 DIA-NN traces in a sample (dossier §3.2). It is a DIA-NN rule choice, and its quadratic describes the true mass error worse than a constant does.

4. Which rules are right? (re-extraction from the raw centroids, 3,000 IDs)

How this was assessed. There is no ground truth for a trace, so the rules were judged on the centroids themselves. For a random sample of 3,000 DIA-NN-identified precursors of run 01, every centroid within ±25 ppm of each fragment and MS1 isotope target was recorded at every point of the trace (`scripts/quality_reextract.py`, reading the mzML directly). A mass-error model was fitted to the apex centroids of the top-6 fragments of the confident IDs: a mean μ(m/z, intensity), quadratic in m/z with an upward drift for strong centroids, and a width σ(intensity) = √(0.83² + 70²/I) ppm, i.e. 2.4 ppm at 10^3 and 0.86 ppm at 10^5. A centroid is counted as true signal when it lies within 3σ(I) of μ. Each engine's rule (window centre, width, max or sum) is then applied to the recorded centroids; **capture** is the share of the true signal in the peak region (apex ±4 cycles) that the rule returns, **interference** the share of what it returns that is not true signal (an upper bound, since the tails of the error distribution hold real signal too). Shape metrics (fragment co-elution, library cosine at the apex, dropouts) are computed on the same re-extracted traces. The rule emulation was validated first: for every engine it reproduces the engine's own exported traces with a median r of 1.0 (`scripts/quality_sample.py`, `results/analysis/quality/`; details in `analysis/trace_quality.md`).

MS2 (run 01; DIA-NN's accuracy ±5.9 ppm; ours A at the V0.2 defaults: 9σ = ±7.9 ppm around the measured offset with its m/z term):

rule	capture	interference	co-elution	cosine at apex	dropouts
ours A (V0.2 defaults, max, ±7.9 ppm)	**99.9%**	4.7%	0.521	0.920	14.8%
DIA-NN (quadratic centre, max, ±5.9 ppm)	99.5%	2.8%	0.522	0.919	14.9%
ours B = OpenSWATH (no correction, sum, ±5.9 ppm)	97.3%	3.1%	0.467	0.903	16.9%

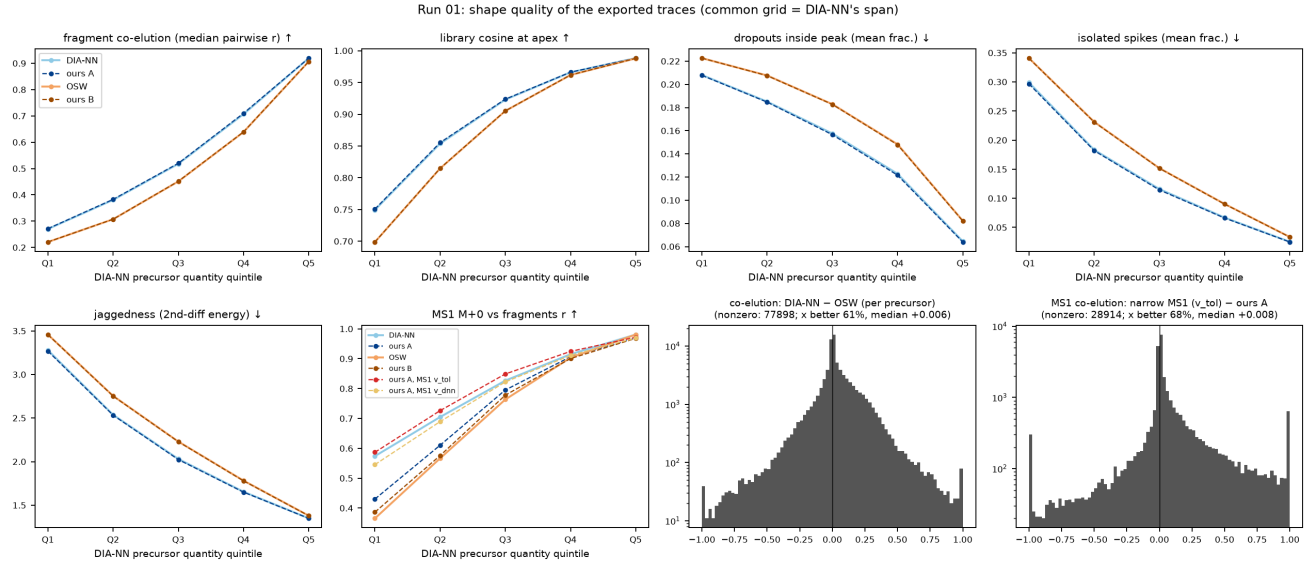
Ours A and DIA-NN give the same trace shapes; A's wider window trades 1.9 points of interference for the last 0.4% of signal (at ±5.9 ppm around our offset it is 99.6% / 2.9%, indistinguishable from DIA-NN). B's loss is its centre: with no correction, the ±5.9 ppm window ends only 2.6 ppm beyond the true −3.2 ppm offset. Max versus sum is irrelevant on Astral MS2: at most 4×10^{-5} of the points hold two centroids in one window.

MS1, M+0 (peak region apex ±6 MS1 scans; the truth model here is 3σ(I) around the MS1 median, σ 0.85 ppm):

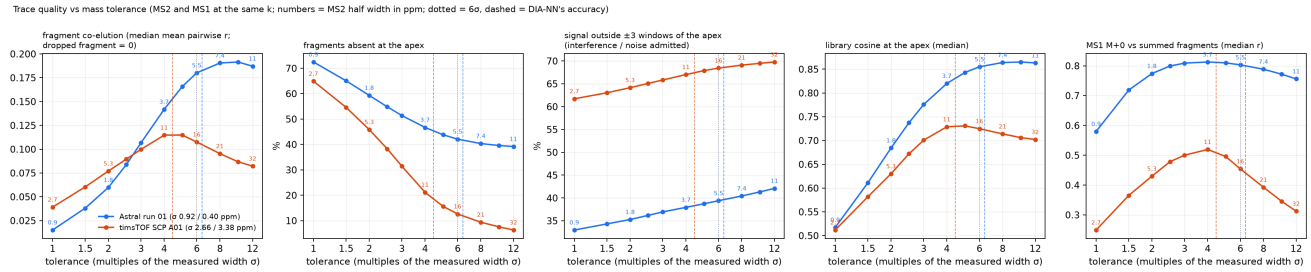
rule	capture	interference
ours A (V0.2 defaults: measured offset with m/z term, max, 4σ = ±1.43 ppm)	92.6%	**0.0%**
ours A at DIA-NN's width (measured offset, ±2.13 ppm)	**98.0%**	0.9%
DIA-NN export (uncorrected m/z, max, ±2.13 ppm)	91.6%	5.4%
ours B = OpenSWATH (no correction, sum, ±5.9 ppm)	100%	22%

The corrected centre is what matters: at DIA-NN's own width it captures 98% at 0.9% interference where DIA-NN's uncorrected export captures 92% at 5.4%; the wide OpenSWATH window admits a quarter of its peak intensity as interference. The 4σ default is on the narrow side by this truth model: the width measured on the anchors (0.36 ppm) comes from strong apex centroids, while weak MS1 centroids scatter more (§5 of `EXTRACTION.md`, the intensity dependence), so ±1.43 ppm leaves 7% of the modelled true signal outside. MS1-to-fragment co-elution on the exports nevertheless ranks ours A (0.886) above DIA-NN (0.880) and OpenSWATH (0.826). A 5–6σ MS1 default is the candidate change this points at; it is left for the next release with the tolerance work. OpenSWATH additionally places

isotope k at m/z + k·1.00336 without dividing by the charge, so its M+1 and M+2 traces of charge ≥ 2 precursors are a different ion.



5. Trace quality as a function of the tolerance (both instruments)



The DIA-NN-identified precursors extracted at k × σ, k = 1 … 12, and scored on the traces (`scripts/tolerance_screen.py`):

	MS2 co-elution optimum	MS1 optimum	DIA-NN's own accuracy
Astral run 01	k = 8–10 (±7.4–9.2 ppm)	k = 4 (±1.6 ppm)	6.4σ (±5.9 ppm)
timsTOF A01	k = 4–5 (±10.6–13.3 ppm)	k = 4 (±13.5 ppm)	4.4σ (±11.8 ppm)

On Astral the curve saturates (heavy error tail, dropouts keep falling); on timsTOF it peaks and falls (Gaussian errors, interference costs beyond 5σ). DIA-NN's optimised accuracy sits at each instrument's optimum; MS1 peaks at 4σ on both.

Can the calibration find the multiple by itself? No. Every anchor-based criterion tried rises monotonically to 8σ on timsTOF too: the all-fragment co-elution curve of the anchors, the same on the weaker half of the anchors, on their weak fragments only, and on both; and the anchor yield at a 1% decoy-calibrated false rate, which the shifted-fragment decoys make meaningless (0–5 decoys per run). Anchors are present precursors whose fragments stay correlated as the window widens; the screen's optimum comes from interference in weak and absent precursors, a population the calibration does not have. **Decision:** the multiples are set per instrument class from the run's metadata (MS2 9σ Orbitrap/Astral, 4σ timsTOF and other TOF, 6σ with a warning when unknown; MS1 4σ), as explicit options (`-mz_sigma_ms2`, `-mz_sigma_ms1`; `-mz_tol*` for fixed ppm). A measured criterion needs a population containing absent peptides, i.e. peptide decoys from the library generator (backlog).

6. A second instrument: timsTOF SCP diaPASEF

Runs A01 and B01 go through the mzPeak reader and through DIA-NN 1.7.12 via our mzML export. Calibration finds 775 anchors (25 ng sample), MS2 width 2.6–2.7 ppm, MS1 2.2 ppm with the m/z-dependent offset that this instrument's MS1 needs (+15 ppm per 1000 Th; `EXTRACTION.md`).

How the ion-mobility dimension is handled. A diaPASEF MS2 frame is one spectrum that covers several isolation windows, each active over its own 1/K0 band, and holds the centroids of every TIMS scan, so one ion appears once per scan it spans. The reader uses the mobility in two steps and then integrates it out: (1) each centroid is assigned to the isolation window whose mobility band contains it (the bands come from the frame's precursor descriptions; with the `precursor_index` quirk of the converter the ions are matched to windows by m/z), which restores the window selectivity that a flat read of the frame would lose; (2) within a window the scans are collapsed: centroids within 15 ppm of each other are summed and placed at their intensity-weighted m/z. The grid then holds one ordinary spectrum per window and cycle, and the extraction is the same as on the Orbitrap. MS1 frames are collapsed over the whole mobility range in the same way. The library's predicted CCS/1/K0 column is not used: no mobility filter is applied to a precursor's trace, so the trace integrates every mobility at which its fragments appear inside the window's band. The same frames, collapsed the same way, are what the mzML export hands to DIA-NN, so the comparison below is between extraction rules, not between mobility treatments. Mobility as an extraction axis (filtering a precursor's fragments to its predicted 1/K0, or returning mobility-resolved traces) is the obvious next step for this instrument class and is not part of V0.2.

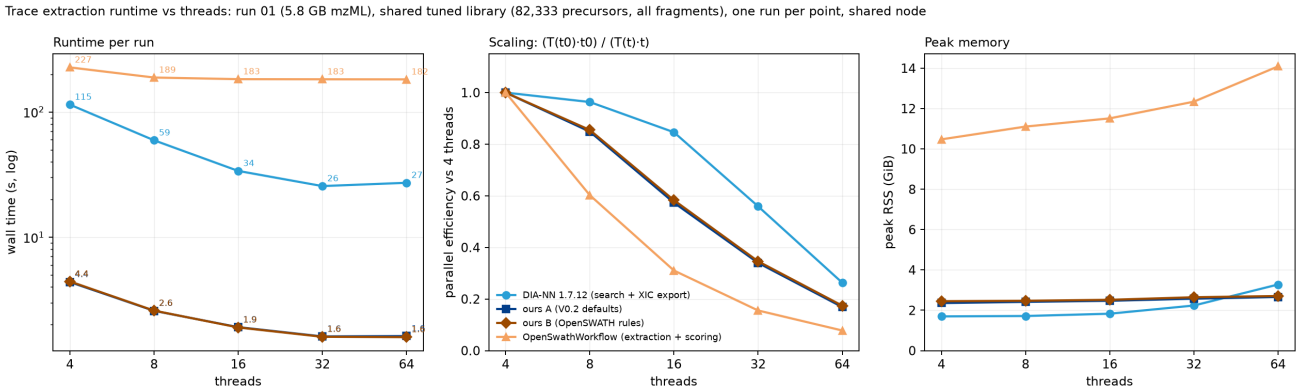
ours vs DIA-NN (its IDs)	MS2 median r	MS2 r > 0.9	MS1 r > 0.9
A01, defaults (4σ)	0.992	86.0%	63%
A01, at DIA-NN's ±11.8 ppm	0.9993	91.7%	60%
B01, defaults (4σ)	0.974	76.0%	67%
B01, at ±11.8 ppm (DIA-NN: ±12.6)	0.996	89.4%	66%

At equal tolerance the engines agree on diaPASEF as on Astral. **MS1 agreement of 60–67% looked like a defect and is not:** where both engines have signal the values are identical, and each has zeros where the other has signal, so the two windows sit at different centres. Placing our MS1 window at the uncorrected library m/z with DIA-NN's MS2 accuracy raises the agreement to 79% (widening to ±20 ppm drops it to 43%): DIA-NN's XIC export takes MS1 at the uncorrected m/z with its MS2 accuracy, the rule already found on Astral (§4), where the 1.3 ppm MS1 offset hides it; here the offset is 3.4 ppm plus the m/z slope. Our MS1 traces are the mass-corrected ones.

7. Performance

Machine: 2 × AMD EPYC 9654 (56 cores per socket online, SMT), shared node; inputs on local `/scratch`; one run per point unless stated; peak RSS from `/usr/bin/time`.

Runtime vs threads, run 01, shared tuned library (82,333 precursors, all fragments); ours at the V0.2 defaults (commit cb466f0), DIA-NN and OpenSWATH from the same series with unchanged binaries:



threads	ours A (defaults)	ours B (OpenSWATH rules)	DIA-NN 1.7.12	OpenSwathWorkflow
4	4.4 s / 2.4 GiB	4.4 s / 2.4 GiB	115 s / 1.7 GiB	227 s / 10.5 GiB
8	2.6 s	2.6 s	59 s	189 s
16	1.9 s	1.9 s	34 s	183 s
32	1.6 s	1.6 s	26 s	183 s
64	1.6 s / 2.7 GiB	1.6 s / 2.7 GiB	27 s / 3.3 GiB	182 s / 14.1 GiB

Ours is 21–26× faster than DIA-NN and 52–114× faster than OpenSWATH at every thread count (both references do more: DIA-NN searches, OpenSWATH scores). Ours flattens beyond 32 threads at this library size (1.6 s, of which ingest 0.3 s and calibration about 0.5 s); OpenSWATH stops scaling at 8 threads while its memory grows to 14 GiB. The node carried a load of 13–16 from other users during the series (`results/threads_cb466f0`).

Full libraries, run 01, 64 threads, top 6 fragments, V0.2 defaults:

library	total (2 runs)	calibration	extraction	peak RSS	traces
human, 3.49 M precursors	5.8–6.1 s	0.5 s	3.3–3.4 s	3.9 GiB	31.3 M
entrapment, 7.0 M precursors	9.3 s	1.0 s	6.1–6.2 s	4.8 GiB	63.0 M

Measured at a node load of about 21; the same binary measured 8.6–9.0 s on the entrapment library on a quieter node (dossier §4). Output is byte-identical for 1 and 64 threads and across the three readers.

diaPASEF (A01, 4.15 M-precursor library, 24 threads): 45 s wall, 13.8 GB peak RSS, 36.7 M traces; the mzPeak per-spectrum read and the mobility collapse take 20–26 s of it, the calibration 4 s, the extraction 12–13 s.

What the speed rests on (dossier §4, `06-parallel-efficiency.md`, `07-contention-diagnosis.md`): one mmap'd peak pool the parsers decode straight into; the library streamed in parts, never resident; one merge join per spectrum with a bucket index over m/z; a central scheduler with four task classes and an ordered commit; recycled trace and Arrow buffers with a per-thread cache; 32 Parquet parts written by their own tasks. Each step was accepted only with a byte-identical trace hash. The library-load transient (decoded row groups, parts and the joined copy coexisting) and the mzPeak reader's serialised decode (scales to 10–14 cores) are the known remaining costs.

8. Reproduction

- Code: https://github.com/okohlbacher/dia-trace-extract-prototype, release tag `V0.2` (this report is attached to the release); build instructions in Appendix A, data sources in Appendix B, command lines in Appendix C.
- Benchmarks: `scripts/benchmark_refined.sh` (three-way comparison on a shared recalibrated library), `scripts/bench_threads.sh` (thread series), `scripts/compare_traces.py` (pairwise metrics),

`scripts/tolerance_screen.py` and `plot_tolerance_screen.py` (§5), `scripts/plot_thread_scaling.py` (§7), `scripts/doc_figures.py` (§2 figures). Mass-error dumps: `ODIA_DUMP_MASS_ERRORS=file`.

- Inputs: Appendix B; libraries in `library/` (DIALibGen 0.11.1, `library/generate.sh`, `library/hye_timstof.log`).
- Tests: `ctest` in the build directory (7 tests: TraML extraction, OpenSWATH parity, byte-level Parquet compare).

Appendix A: repository and installation

Repository. https://github.com/okohlbacher/dia-trace-extract-prototype (tag `V0.2`, commit 663dc39 and the report commits after it). The mzPeak reader is the submodule `external/mzpeak` (okohlbacher/mzpeak-openms, tag `partitioned-read-2026-10-11`); the library generator is okohlbacher/DIALibGen (0.11.1; the build fix for current OpenMS develop is on its branch `compact-parquet-plain-strings`).

Requirements. Linux, GCC ≥ 13, CMake ≥ 3.24, Ninja; an OpenMS develop (≥ 3.7) build tree with its vcpkg dependencies, which provide Arrow/Parquet and, with `boost-json` and `boost-program-options` added to the OpenMS vcpkg manifest, the Boost ≥ 1.89 the mzPeak library needs (another `BoostConfig.cmake` directory can be given with `-DMZPEAK_BOOST_DIR`). Python 3 with pyarrow, pandas, numpy and matplotlib (`tools/pyenv` here) for the scripts; `markdown` and `weasyprint` for the PDF.

```
 git clone https://github.com/okohlbacher/dia-trace-extract-prototype.git && cd dia-trace-
extract-prototype
git checkout V0.2
git submodule update --init
# OpenMS develop, built with its vcpkg manifest (here at OpenMS/OpenMS-build via `cmake --
preset linux-x64-release`)
O=$PWD/OpenMS/OpenMS-build
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH="$O;$O/
vcpkg_installed/x64-linux-dynamic"
cmake --build build -j 16
ctest --test-dir build        # 7 tests: TraML extraction, OpenSWATH parity, byte-level Parquet
compare
```

Use.

```
 # defaults: RT map, mass offsets and tolerances measured from the run; +-60 s around the
library RT
# (-rt_mode minutes: tuned libraries carry run minutes; auto: predicted RT, the map is fitted
from the run)
build/DIATraceExtractor -in run.mzML -lib library.parquet -rt_mode minutes -threads 64 \
  -out traces.parquet -out_stats stats.json -out_calibration anchors.tsv
build/DIATraceExtractor -in run.mzpeak -lib library.parquet -rt_mode auto -threads 64 -out
traces.parquet
build/DIATraceExtractor -in run.mzpeak -out_mzml run.mzML            # the loaded run as
centroided mzML, for other tools
build/DIATraceExtractor --help                                       # all options, including
calibration:*
```

Input formats: mzML (indexed, centroided) and mzPeak; libraries as DIALibGen compact Parquet, DIA-NN/OpenSWATH TSV, TraML or PQP. The benchmark inputs come from the ProteoBench DIA Astral entrapment and diaPASEF modules (https://proteobench.cubimed.rub.de/raws/DIA-astral-entrapment/); the reference engines are DIA-NN 1.7.12 (the last open-source version, with its `--xic` export patched: `tools/DiaNN-1.7.12/`) and OpenSwathWorkflow from the same OpenMS build.

Appendix B: data sets

Orbitrap Astral (ProteoBench DIA ion entrapment module). Three single-species human runs of the "LFQ Benchmark Dataset - Generation Beta" (Van Puyvelde et al., Dhaenens lab, Ghent;

ProteomeXchange **PXD070049**, https://www.ebi.ac.uk/pride/archive/projects/PXD070049): `LFQ_Astral_DIA_15min_50ng_Human_01/02/03.raw`, Orbitrap Astral, 15-min gradient, 50 ng. ProteoBench mirrors them with the module's search database:

file	source
`LFQ_Astral_DIA_15min_50ng_Human_01.raw` (3.1 GB), `_02.raw`, `_03.raw`	https://proteobench.cubimed.rub.de/raw DIA-astral-entrapment/ (also `all_data_Entrapment_DIA_Astral.ta` 5.6 GB)
`ProteoBenchFASTA_Entrapment_Human_with_contaminants_entrapment_pep.zip` (2,841,736 peptide-level entries: human targets, contaminants and entrapment peptides, 1:1)	https://proteobench.cubimed.rub.de/fas
`UP000005640_9606.fasta` (UniProt human reference proteome, 20,652 entries; the human library)	https://www.uniprot.org/proteomes/UP000005640

The `.raw` files were converted to centroided, indexed mzML with OpenMS `FileConverter` from the same OpenMS develop build (Thermo raw read through its ThermoRawFileParser bridge, vendor centroiding; `benchmark-data/DIA-astral-entrapment/mzML/convert_*.log`); mzPeak variants were made with mzpeak-convert 0.17.2 / 0.18.0 and with this tool's `MzMLToMzPeak`.

timsTOF SCP diaPASEF (ProteoBench DIA quantification, precursor-ion level, diaPASEF module). Three-species mixture (human, yeast, E. coli commercial digests in the proportions of Van Puyvelde et al., 2022, Scientific Data 9:126, https://www.nature.com/articles/s41597-022-01216-6; expected log2 A/B: human 0, yeast −1, E. coli +2), 25 ng in triplicate, Bruker timsTOF SCP with an UltiMate 3000 RS nanoLC and a 25 cm IonOpticks Aurora column, diaPASEF with precursor m/z 400–1000, 1/K0 0.64–1.37, 0.96 s cycle. ProteomeXchange **PXD062685** (announced by the module; at the time of writing the files are served by ProteoBench):

file	source
`ttSCP_diaPASEF_Condition_A_Sample_Alpha_01_11494.d.zip` (7.3 GB), `..._Condition_B_Sample_Alpha_01_11496.d.zip` (7.1 GB); replicates 02/03 of both conditions available, not used	https://proteobench.cubimed.rub.de/raws/diaPASEF/
`ProteoBenchFASTA_MixedSpecies_HYE.zip` (31,889 entries: three proteomes plus contaminants prefixed `Cont_`, Frankenfield et al., J. Proteome Res. 2022)	https://proteobench.cubimed.rub.de/fasta/ProteoBenchFASTA_MixedSpecies_HYE.zip

The `.d` folders were converted with mzpeak-convert 0.18.0 (`benchmark-data/diaPASEF/convert_*.log`); the mzML files DIA-NN read are this tool's `-out_mzml` export of the loaded grid.

Reference software. DIA-NN 1.7.12 (https://github.com/vdemichev/DiaNN, release 1.7.12, the last open-source version; built here with a one-line fix to its `--xic` export, `tools/DiaNN-1.7.12/`); OpenSwathWorkflow from the OpenMS develop build used for the tool itself; DIALibGen 0.11.1 (https://github.com/okohlbacher/DIALibGen) with AlphaPeptDeep ONNX models for the predicted libraries.

Appendix C: command lines

Paths: `$P` = the repository, `$S` = `/scratch/$USER/odia-bench` (inputs staged from `benchmark-data/` and `library/`), `$X` = `build-stream/DIATraceExtractor` at the commit named, `$DIANN` = `tools/DiaNN-1.7.12/diann-1.7.12-xicfix`, `$OSW` = `OpenMS/OpenMS-build/bin/OpenSwathWorkflow`. The libraries in each tool's format come from `scripts/lib_to_diann17_tsv.py` (`--rt_scale 1`) and `scripts/lib_to_openswath_tsv.py` (`--rt_scale 60`). DIA-NN's XICs are converted with `scripts/diann17_xic_to_parquet.py`, OpenSWATH's with `TextExporter -xic:mode explode`; the pairwise metrics come from `scripts/compare_traces.py`.

Astral, three-way comparison on the shared tuned library (§3, §4; `scripts/benchmark_refined.sh 01|02|03 64`, binary cbad9dd, V0.1 rule for ours A). Run 01 shown; runs 02 and 03 differ only in the run and the library's `_ids_runNN` suffix.

```
# DIA-NN 1.7.12, round 1 on the predicted library (gives the IDs that restrict the shared
library), then round 2 with XICs
$DIANN --f $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML --lib library/diann17/
proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01.tsv \
       --out report.tsv --temp temp --threads 64 --verbose 1 --qvalue 0.01
$DIANN --f $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML --lib lib_diann17.tsv --out
report.tsv --temp temp \
       --threads 64 --verbose 1 --qvalue 0.01 --xic 60
# ours A (V0.1 rule: MS2 +-6 ppm, MS1 measured), ours P (DIA-NN parity: both tolerances = DIA-
NN's 5.9051 ppm), ours B (OpenSWATH rules)
$X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib library/refined/
proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01_ids_run01.parquet \
   -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol 6 -mz_tol_ms1 0 -aggregation max -
mass_calibration auto -threads 64 -out odiaA.parquet -out_stats odiaA.json
$X ... -mz_tol 5.9051 -mz_tol_ms1 5.9051 -aggregation max -mass_calibration auto -threads 64 -
out odiaP.parquet
$X ... -mz_tol 5.9051 -mz_tol_ms1 5.9051 -aggregation sum -ms1_isotope_spacing unit -threads 64
-out odiaB.parquet
# OpenSwathWorkflow (full window = 2 x DIA-NN's accuracy)
$OSW -in benchmark-data/DIA-astral-entrapment/mzML/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -tr
lib_openswath.tsv -out_chrom osw.xic -out_features osw.featureXML \
     -rt_extraction_window 120 -mz_extraction_window 11.8102 -mz_extraction_window_unit ppm -
mz_extraction_window_ms1 11.8102 -mz_extraction_window_ms1_unit ppm \
     -ms1_isotopes 2 -Calibration:auto_irt:enabled false -threads 64
```

Astral, runtime vs threads (§7; `ODIA_ONLY=1 REF=results/threads scripts/bench_threads.sh results/threads_cb466f0 4 8 16 32 64`, binary cb466f0). `$t` = 4, 8, 16, 32, 64; DIA-NN and OpenSWATH as above with `--threads $t` / `-threads $t` (`results/threads/`, unchanged binaries).

```
# ours A: the V0.2 defaults (tolerances by instrument class, m/z-dependent offset when
material)
$X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib $S/input/
proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01_ids_run01.parquet \
   -rt_mode minutes -rt_window 60 -top_fragments 0 -threads $t -out $S/out/t$t/odiaA.parquet
# ours B
$X ... -rt_mode minutes -rt_window 60 -top_fragments 0 -mz_tol 5.9051 -mz_tol_ms1 5.9051 -
aggregation sum -ms1_isotope_spacing unit -threads $t -out $S/out/t$t/odiaB.parquet
```

Astral, full libraries (§7; `results/full_cb466f0/`, binary cb466f0, top 6 fragments, 2 runs each).

```
$X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib library/
human_UP000005640_astral_nce25_v0.11.1_tuned_run01.parquet -rt_mode minutes -threads 64 -out
$S/out/full_human.parquet
$X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib $S/input/
proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01.parquet -rt_mode minutes -threads 64 -
out $S/out/full_entrap.parquet
```

Astral, mass-error structure and the linear-vs-constant check (`EXTRACTION.md`, mass model; binary 4e89acc and its parent).

```
ODIA_DUMP_MASS_ERRORS=astral01.tsv $X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib
$S/input/proteobench_entrapment_astral_nce25_v0.11.1_tuned_run01.parquet -rt_mode minutes -
threads 48 -out ...
$X -in $S/input/LFQ_Astral_DIA_15min_50ng_Human_01.mzML -lib library/refined/
proteobench_entrapment_astral_nce25_v0.11.1_shared_run01.parquet -rt_mode minutes -ids results/
tolscreen/astral_run01_ids.txt -threads 32 -out astral01_lin.parquet
$X ... -calibration:mass_model constant -out astral01_const.parquet
```

Tolerance screen (§5; `scripts/tolerance_screen.py`, binary 4934ea5). The script runs the extractor once per k with `-mz_tol k*sigma_ms2 -mz_tol_ms1 k*sigma_ms1 -top_fragments 0 -ids ...` and scores the traces.

```
 tools/pyenv/bin/python scripts/tolerance_screen.py --run $S/input/
LFQ_Astral_DIA_15min_50ng_Human_01.mzML \
    --lib library/refined/proteobench_entrapment_astral_nce25_v0.11.1_shared_run01.parquet --
ids results/tolscreen/astral_run01_ids.txt \
    --sigma-ms2 0.92 --sigma-ms1 0.40 --rt_mode minutes --out results/tolscreen/astral_run01
tools/pyenv/bin/python scripts/tolerance_screen.py --run $S/input/diapasef/A01.mzpeak --lib
library/hye_timstof.parquet --ids results/diapasef/diann_A01_ids.txt \
    --sigma-ms2 2.66 --sigma-ms1 3.38 --rt_mode auto --out results/tolscreen/timstof_A01
tools/pyenv/bin/python scripts/plot_tolerance_screen.py docs/figures/tolerance_screen "Astral
run 01=results/tolscreen/astral_run01.tsv:6.4" "timsTOF A01=results/tolscreen/timstof_A01.tsv:
4.4"
```

timsTOF diaPASEF (§6; `results/diapasef/`). `.d` to mzPeak with mzpeak-convert 0.18.0; the mzML for DIA-NN is this tool's export of the loaded grid; `B01` identical with the B-condition files.

```
 /ceph/ibmi/abi/oliver/bin/mzpeak-convert ttSCP_diaPASEF_Condition_A_Sample_Alpha_01_11494.d -o
A01.mzpeak
$X -in $S/input/diapasef/A01.mzpeak -out_mzml $S/input/diapasef/A01.mzML -threads 32
# frames split per window, TIMS scans collapsed within 15 ppm
$DIANN --f $S/input/diapasef/A01.mzML --lib library/diann17/hye_timstof.tsv --out report.tsv --
temp temp --threads 32 --verbose 1 --qvalue 0.01 --xic 60
# ours, all precursors (cost figures; `-out_calibration` for the anchors), then on DIA-NN's IDs
at the defaults (4 sigma) and at DIA-NN's accuracy
$X -in $S/input/diapasef/A01.mzpeak -lib library/hye_timstof.parquet -rt_mode auto -threads 32
-out odia_A01.parquet -out_stats odia_A01.json -out_calibration odia_A01.anchors.tsv
$X -in $S/input/diapasef/A01.mzpeak -lib library/hye_timstof.parquet -rt_mode auto -ids
results/diapasef/diann_A01_ids.txt -threads 32 -out odia_A01_idsdef.parquet
$X ... -ids results/diapasef/diann_A01_ids.txt -mz_tol 11.8 -mz_tol_ms1 11.8 -threads 32 -out
odia_A01_idsP.parquet
# MS1 window probe around our offset (+-4, 7, 20, 30 ppm) and the uncorrected-centre test that
explains DIA-NN's MS1 export
$X ... -ids results/diapasef/diann_A01_ids.txt -mz_tol 11.8 -mz_tol_ms1 7 -out
odia_A01_ms1_7.parquet
$X -in $S/input/diapasef/A01.mzpeak -lib library/hye_timstof.parquet -rt_mode auto -
mass_calibration off -mz_offset -4.6 -mz_tol 10.8 -mz_offset_ms1 0 -mz_tol_ms1 11.8 \
   -ids results/diapasef/diann_A01_ids.txt -threads 24 -out timstof_A01_ms1raw11.8.parquet
```

Libraries. `library/generate.sh human|entrapment` (DIALibGen 0.11.1, Astral NCE 25); the timsTOF library as recorded in `library/hye_timstof.log`; the tuned RT models with `DIALibGen -mode tune` on DIA-NN's run-01 IDs (`library/*.log`); the `_ids_runNN` and `_shared_runNN` libraries with `scripts/filter_library.py` on DIA-NN's report (`scripts/benchmark_refined.sh`, step 2).